3.2 Preventing Address Guessing

This section introduces *twin allocation*, a technique that allows our model to prevent a program from guessing addresses of objects without data-flow provenance tracking.

A problem with wildcard provenance is that a pointer formed out of an integer can access any object. Consequently, a program may be able to guess the address of any object and access it. This makes precise alias analysis very difficult.

A simple idea to prevent guessing is to exploit the fact that allocation functions return a non-deterministic value:

```
char *p = malloc(4);  // (val=*, obj=p)
char *q = 0x10;
*q = 0; // UB if val(p) != 0x10
```

This program can guess the address of `p` in a possible execution where `malloc` returns `0x10`. However, there is at least one execution where the program fails to guess the address of `p` (e.g., `malloc` returns `0x20`), and so the program would trigger UB. Since there is at least one execution where the program would trigger UB, the compiler can assume `q` cannot alias with `p` (or with any object at all).

Even with non-deterministic allocations, a program can still (desirably) observe the address of an object such that a pointer created by casting from an integer can alias with that object, e.g.:

```
char *p = malloc(4);  // (val=*, obj=p)
*p = 0;
int v = 0x10;
if ((int)p == v)
    *(int*)v = 1;
print(*p); // can print 0 or 1
```

This program is well-defined and may print `0` or `1`, depending on the return value of `malloc`. The comparison is sufficient for the program to observe the address of object `p`, and so `*(int*)v = 1` will not trigger UB in any execution.

However, this semantics still has a caveat: it allows programs to guess addresses through a "side-channel leak." The leak happens when the memory only has a single address left where a new object can be allocated. For example, assume that a system has an 8-bit heap segment as well as 8-bit pointers, that heap address `0x00` is legal, and that the allocations below succeed:

```
char *p = malloc(0x80);
char *q = malloc(0x80);

*q = 0;
int v = ((int)p == 0x00) ? 0x80 : 0x00;
*(char*)v = 1;

print(*q); // prints 1
```

Since each heap cell is half the size of the address space, there are only two possible heap configurations: `p`-first or `q`-first. Therefore, a single test allows the program to guess the address of `q` without having to explicitly observe it. In other words, when memory is finite, returning a non-deterministic value from allocation functions is not sufficient to prevent programs from guessing addresses.